

Foundations of R Chapter 4

Robert Gulotty

9/30/2018

The Data Science Workflow

In this chapter, we will formally introduce the data science workflow. This means pipes, join commands, and the other tools of getting from raw data to a usable dataset. We will emphasize tools that scale with larger datasets, enable more replicable code, and are less likely to produce significant errors.

A key lesson of this chapter is that robustness requires transparency. While quantitative research is no more subject to errors than other approaches, the power of the claims can make even small errors problematic. In addition, many steps of the quantitative process can be done transparently, enabling criticism and scientific progress.

This chapter walks through data analysis using a linear model. We begin with a brief return to the flights data before turning to a substantive analysis of cross-country data on changes in religious identification. The last of these will introduce some of the tools of regression, which we will cover in detail.

The Length of flights

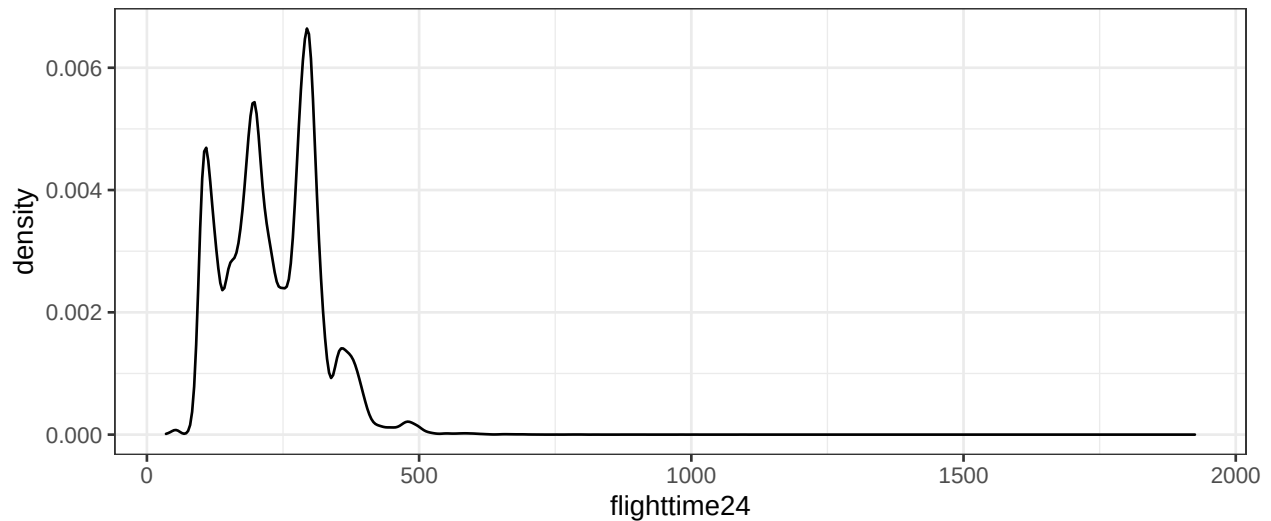
We begin with a brief and final discussion of the flights data. Recall we found that there were some problems with flight times, namely that there is an issue of time zones in the destination. Recall the following operations:

```
flightscomparison <- mutate(flights,
                             flighttime = arr_time - dep_time,
                             flighttime24 = ifelse( flighttime < 0, flighttime+2400, flighttime))
```

Recall we found one hour peaks in the density for the flights plots, such as the following.

```
ggplot(flightscomparison, aes(flighttime24))+geom_density()+theme_bw()
```

```
## Warning: Removed 8713 rows containing non-finite values (stat_density).
```



One of the things that is going on here is the time zone changes. Luckily, there is a dataset of the location of each of the airports in the `nycflights13` package, called `airports`. Here we will join the airports data to the flights data to correct for the timezone.

How to Join Data

If two dataframes are identical in terms of columns, they can be combined with a `bind_rows`. If they are identical in terms of rows, they can be combined with `bind_columns`. If they are neither, we can still combine datasets if there is shared information, an index columns or possibly multiple columns, that share values across each dataset.

```
names(airports)
```

```
## [1] "faa" "name" "lat" "lon" "alt" "tz" "dst" "tzone"
```

```
names(flightscomparison)
```

```
## [1] "year" "month" "day" "dep_time"
## [5] "sched_dep_time" "dep_delay" "arr_time" "sched_arr_time"
## [9] "arr_delay" "carrier" "flight" "tailnum"
## [13] "origin" "dest" "air_time" "distance"
## [17] "hour" "minute" "time_hour" "flighttime"
## [21] "flighttime24"
```

It looks like the `faa` column of `airports` is similar to that of the `origin` and `dest` columns of `flightscomparison`. Because these are all flights originating in New York and have the same time zone, we will join on the destination.

In order to combine the two dataframes, we need a column in each dataframe with the same name. To accomplish this, we add a column to airports using the “mutate” command.

Recall, the default behavior of `%>%` is to place lhs as the first argument, i.e. `x %>% f(y)` is equivalent to `f(x, y)`.

```
airports <- airports %>% mutate( dest = faa)
```

Here we pass the thing on the left to the first argument in the next function. You will see that it makes code much nicer. Consider the following:

```
airportsTZ4 <- filter(airports, tz == -5)

airportsTZ4 <- mutate(airportsTZ4,
                      swardist = sqrt((lat - 40.7)^2+(lon-74)^2))

ggplot(airportsTZ5, aes(swardist, alt))+geom_point()
```

Notice how many times airportsTZ5 is written. We can use the `%>%` operator to save typing.

```
airports %>%

  filter( tz == -5) %>%

  mutate( swardist = sqrt((lat - 40.7)^2+(lon-74)^2)) %>%

  ggplot(aes(swardist, alt))+geom_point()
```

Learning how to use pipes will save time and energy. In general pipe works best with dplyr commands and others that have the input as the first argument of the function. However, it can be used generically. If this purpose you can use the dot (.) as placeholder. `y %>% f(x, .)` is equivalent to `f(x, y)` and `z %>% f(x, y, arg = .)` is equivalent to `f(x, y, arg = z)`.

Left join

Join commands come in several flavors, each corresponding to different assumptions about the relations between two datasets. The most common is the `left_join`, which keeps all the data in the original dataset, and matches the rows it can find in the right.

```
flightsandairports <- flightscomparison%>%left_join(airports)
```

```
## Joining, by = "dest"
```

Notice, that this offers a nice warning to tell you what columns it used to join, but it does not tell you how many rows of airports it dropped to make the final dataset. To do so, we can use the `anti_join`. `anti_join` returns the columns that are not shared. The following code snippet creates two fake datasets, and then joins the data.

```
Administrations <- data.frame(years=2001:2010,  
                              Presidents=c(rep("R", 8), rep("D",2)))  
Troops <- data.frame(years=c(2000, 2001:2004, 2006,  
                            2007, 2009,2010,2012),  
                    troops=c(0,2500,9700,13100,20300,  
                             20300,25000,67000, 100000, 77000))  
AdminTroops<- Administrations%>%left_join(Troops)
```

```
## Joining, by = "years"
```

```
missingTroops<-Administrations%>%anti_join(Troops)
```

```
## Joining, by = "years"
```

```
missingAdmin<-Troops%>%anti_join(Administrations)
```

```
## Joining, by = "years"
```

```
missingTroops
```

```
##   years Presidents  
## 1  2005           R  
## 2  2008           R
```

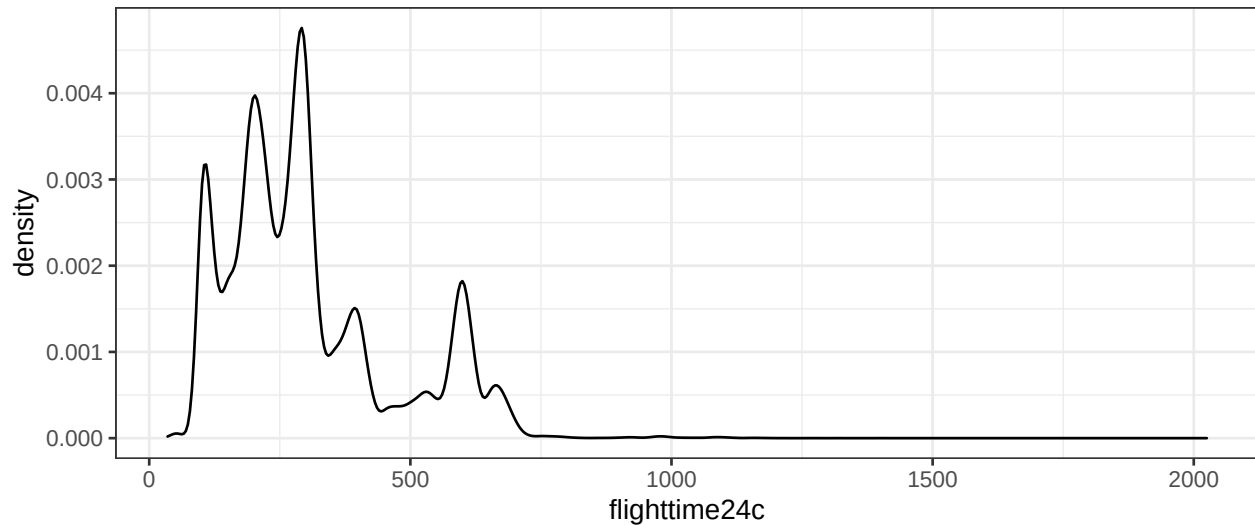
```
missingAdmin
```

```
##   years troops  
## 1  2000       0  
## 2  2012  77000
```

Great, now we can the plot, but correct for the timezone.

```
flightsandairports <- flightsandairports%>%  
  mutate( flighttime24c = flighttime24 - 500 - tz*100)  
  
ggplot(flightsandairports, aes(flighttime24c))+  
  geom_density()+theme_bw()
```

```
## Warning: Removed 16273 rows containing non-finite values (stat_density).
```



Combining data is a powerful tool of analysis, but often data is dispersed across dozens or hundreds of data frames. We have already seen in chapter 2 how loops can perform these repetitive tasks, in later chapters we will learn tools to integrate vectorized code into your work flow to enable analysis at scale.

A join example: Religious adherence over time

Often joins are integrated into an analysis, rather than as a mechanism of preparing data. Here we will analyze changes in the faiths of people across time in the World Religion Dataset, which is in STATA format.

We can read STATA format with the `haven` library:

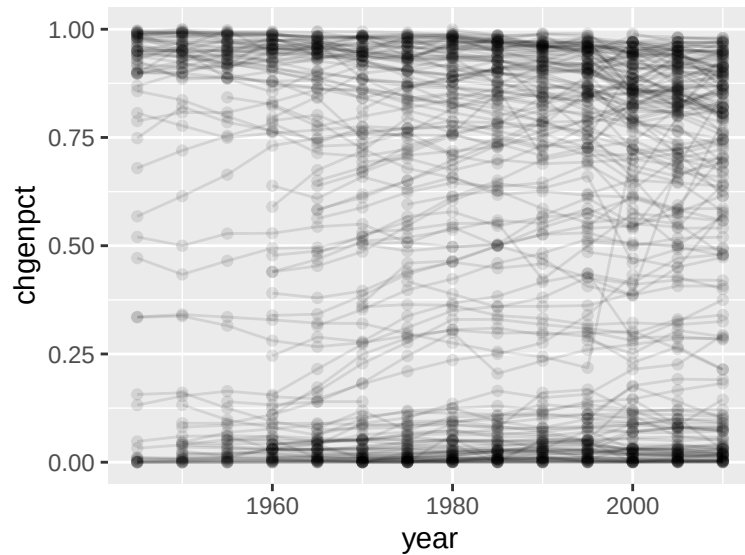
```
library(haven)  
  
religions<-read_dta("~/Dropbox/mathCamp/Code/data/WorldReligion.DTA")  
  
#str(religions)
```

Here we have the percent of Christians (All denominations) by year.

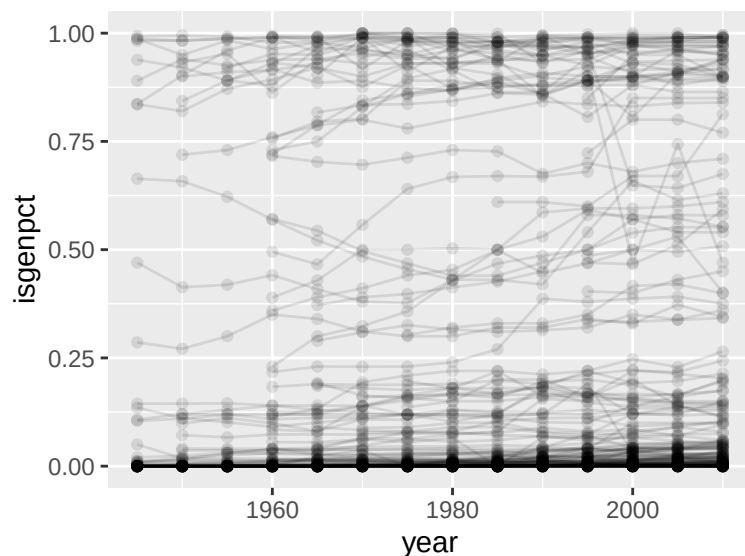
```
ggplot(religions, aes(year,chgenpct))+geom_point()
```


I notice that the years are actually being treated like categorical data, because of some commas in the year variable. Get rid of the commas in the year variable, and tranform it into a numeric data type. We do so with the `gsub()` command.

```
religions <- religions %>%  
  mutate(year = as.numeric(gsub(",", "", year)))  
  
ggplot(religions, aes(year, chgenpct, group = country)) +  
  geom_point(alpha = .1) +  
  geom_line(alpha = .1)
```



```
ggplot(religions, aes(year, isgenpct, group = country)) +  
  geom_point(alpha = .1) +  
  geom_line(alpha = .1)
```



Ok, now we can try to isolate those countries that have a big change. To do so, we group_by country so that when we calculate differences and find the max or min we return that for each country.

```
religionchange <- religions %>%  
  group_by(country) %>%  
  summarise(  
    maxinc = max(c(diff(isgenpct), 0), na.rm = T),  
    maxdec = min(c(diff(isgenpct), 0), na.rm = T)) %>%  
  arrange(maxdec)  
## `summarise()` ungrouping output (override with `.groups` argument)  
  
religionchange <- religionchange %>%  
  mutate(bigchange = ifelse(maxinc > .1 | maxdec < -.1, 1, 0))
```

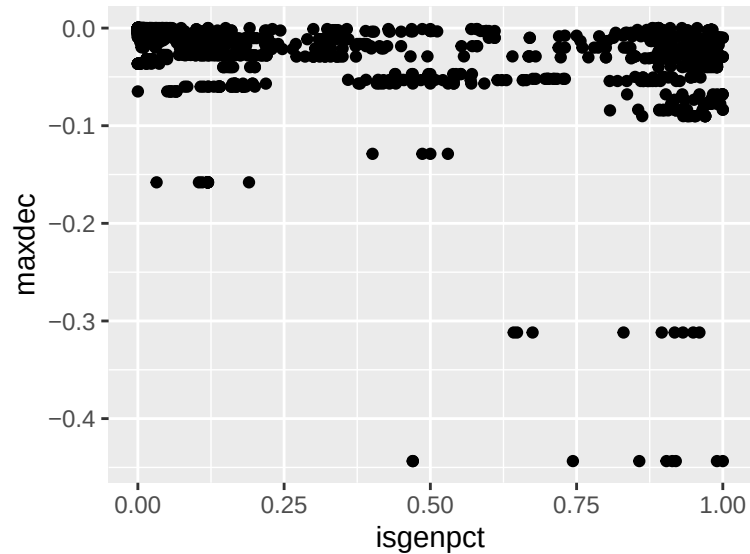

We can recombine our analyzed data this with the original dataset so that we can use it as a variable in plots:

```
religions<-religions%>%left_join(religionchange)
```

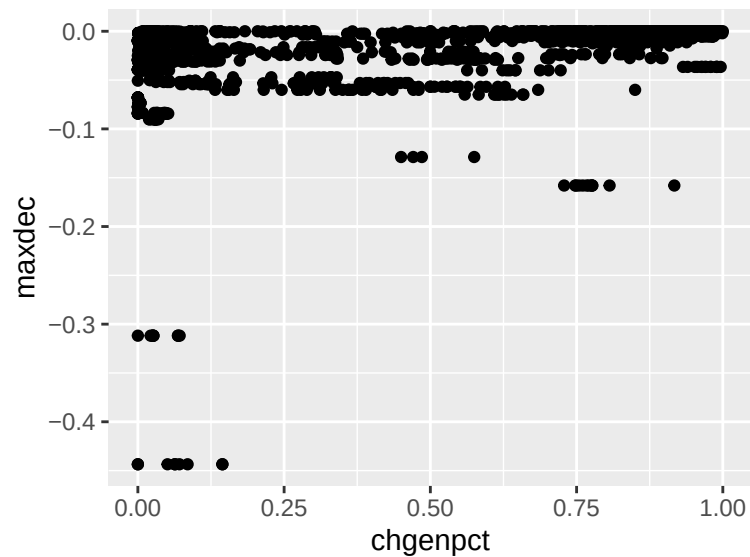
```
## Joining, by = "country"
```

Here we plot the extent that a country's respondents report to follow any form of Islam and the maximum change in the whole dataset, and the same for Christians:

```
ggplot(religions, aes(isgenpct, maxdec, group = "country")) + geom_point()
```



```
ggplot(religions, aes(chgenpct, maxdec, group = "country")) + geom_point()
```



I notice a few outliers, which we can isolate using the filter command, followed by a select command, which identifies columns

```
religions%>%  
  filter(maxdec< -.1)%>%  
  select(year, country, reliab)
```

```
## # A tibble: 34 x 3  
##   year country    reliab  
##   <dbl> <chr>      <dbl>  
## 1  1945 Yugoslvaia 2 [Medium]  
## 2  1950 Yugoslvaia 2 [Medium]  
## 3  1955 Yugoslvaia 2 [Medium]  
## 4  1960 Yugoslvaia 1 [High]  
## 5  1965 Yugoslvaia 1 [High]  
## 6  1970 Yugoslvaia 2 [Medium]  
## 7  1975 Yugoslvaia 1 [High]  
## 8  1980 Yugoslvaia 2 [Medium]  
## 9  1985 Yugoslvaia 2 [Medium]  
## 10 1990 Yugoslvaia 2 [Medium]  
## # ... with 24 more rows
```

One thing I noticed was that the data also includes an indicator for the reliability of the data. It is categorical. Very often we have two categorical data and it is difficult to visualize it. One way to do so is in a table:

```
table(religions$reliab, religions$bigchange)
```

```
##  
##      0      1  
## 1  465    26  
## 2 1196    54  
## 3  248     6
```

Tables are nice, but what we are really looking for is the relative percentages. Is the big change coming along with unreliable data? What we can do is wrap the table in a `prop.table`, which essentially divides by the total of the rows (if we set `margin=1`), or the columns (if we set `margin = 2`).

```
prop.table(table(religions$reliab, religions$bigchange), margin=1)
```

```
##  
##      0      1  
## 1 0.94704684 0.05295316  
## 2 0.95680000 0.04320000  
## 3 0.97637795 0.02362205
```

Whenever reporting such a table, it is best to round the results, and or multiply by 100.

```
round(prop.table(table(religions$reliab, religions$bigchange), margin=1),3)
```

```
##  
##      0      1  
## 1 0.947 0.053  
## 2 0.957 0.043  
## 3 0.976 0.024
```

If you want the `prop.table` to be nicely printed, one option is to use `xtable`, and write the object to html.

```
library(xtable)
```

```
## Warning: package 'xtable' was built under R version 4.0.2
```

```
out.table <-  
xtable(  
  prop.table(  
    table(religions$reliab, religions$bigchange),  
    margin = 1),  
  digits = 2)  
  
print(out.table, type = "html", file = "~/Dropbox/mathCamp/Code/data/religiousreliability.html")
```

There is also the excellent stargazer package.

```
library(stargazer)  
  
stargazer(out.table, summary = F, type = "text")
```

Linear Models

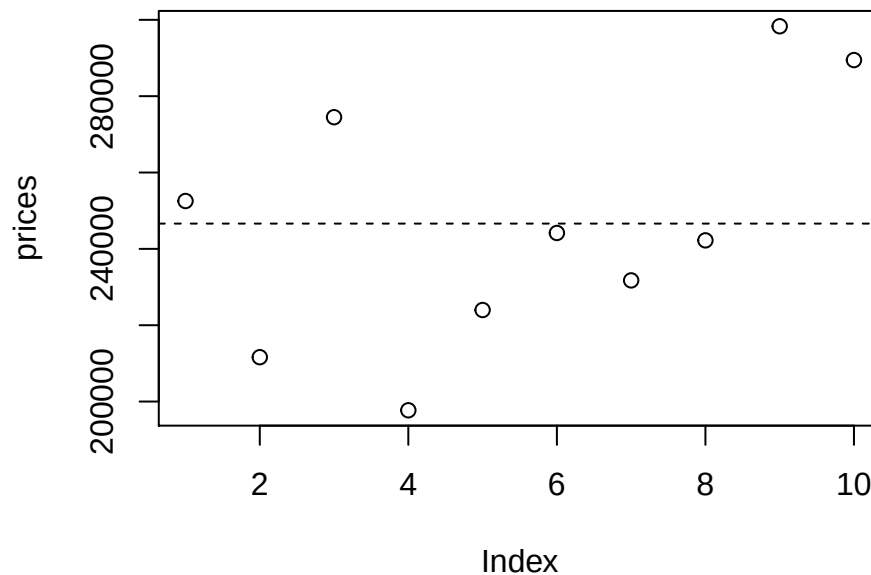
In addition to making a table, we could fit a regression to the data, here we use the “probit” regression, but first, a little background.

When we want to predict the value of something, say Condominium prices, given a set of data, say 10 past purchases, we can use the average of the past prices as our prediction of the future price. Our best guess is the average.

$$E(\text{price}) = \alpha$$

For example, consider the following:

```
prices <- c(252539, 211620, 274491, 197712, 223979, 244161,
           231741, 242225, 298309, 289457)
plot(prices)
abline(h = mean(prices), lty=2)
```



Our best guess for some new point is just the average. In linear model terminology, this would be a model with just the intercept.

```
mean(prices)
```

```
## [1] 246623.4
```

```
summary(lm(prices ~ 1))
```

```
##
```

```
## Call:
```

```
## lm(formula = prices ~ 1)
```

```
##
```

```
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -48911 -20704  -3430   22380   51686
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   246623      10379   23.76 1.98e-09 ***
```

```
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## Residual standard error: 32820 on 9 degrees of freedom
```

Now, suppose that we learn something new, and that is that the Condos are a certain distance from the lake. Now perhaps we want to take advantage of this information, and use the average in three “bins”. Here we take the average for lake distances between 0 and 400, between 400 and 800, and above

```
library(tidyverse)

## -- Attaching packages ----- tidyverse 1.3.0 --
## v tibble 3.0.1      v purrr 0.3.4
## v tidyr 1.1.0      v stringr 1.4.0
## v readr 1.3.1      v forcats 0.5.0

## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()

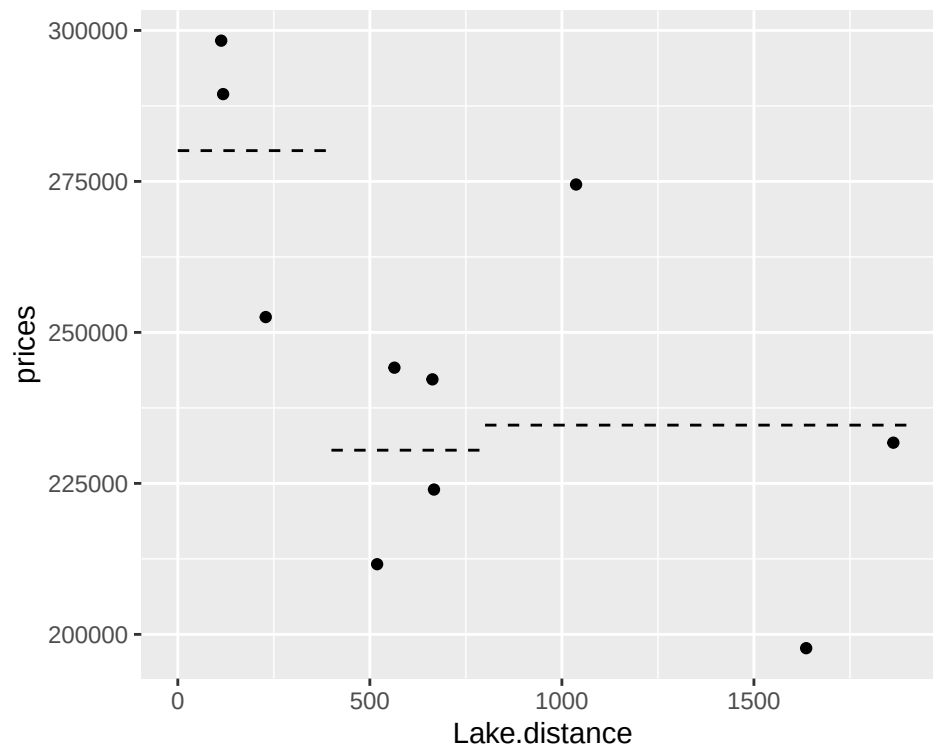
Lake.data= data.frame(
  prices=c(252539, 211620, 274491, 197712, 223979, 244161,
           231741, 242225, 298309, 289457),
  Lake.distance = c(229, 519, 1037, 1636, 667, 564, 1863, 663, 113, 118))

Lake.data<- Lake.data%>%mutate(
  Lake.groups = cut(Lake.distance, c(0,400,900, 1900)))

groupmeans<-Lake.data%>%
  group_by(Lake.groups)%>%
  summarise(mean.prices=mean(prices))%>%
  mutate(cutstarts=c(0,400,800),
  cutends=c(400, 800, 1900))

## `summarise()` ungrouping output (override with `.groups` argument)

ggplot(Lake.data)+
  geom_point( aes(Lake.distance,prices))+
  geom_segment( data=groupmeans,
  aes(x=cutstarts, xend=cutends ,y=mean.prices,yend=mean.prices), lty=2)
```

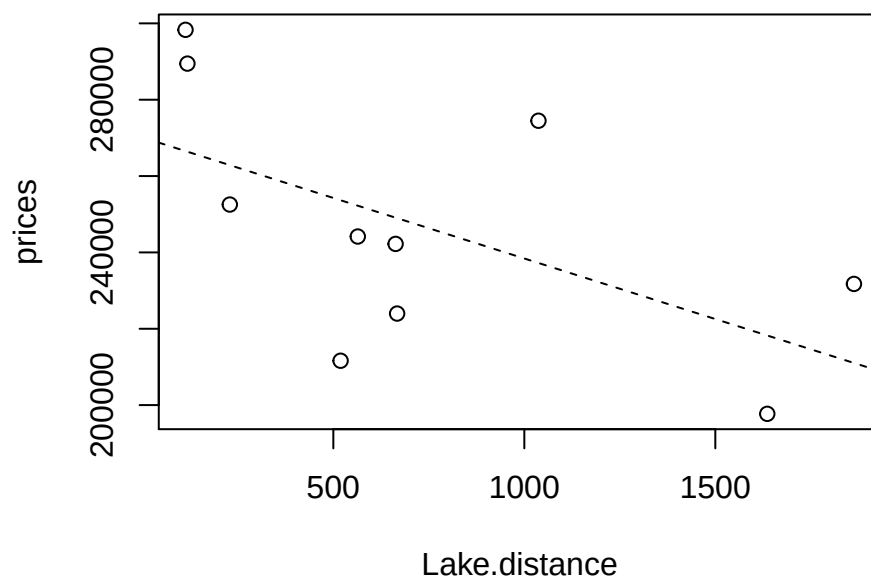


Pretty good, but it seems that the last bin is driven a bit by one point. We might expect that the price is some linear function of the lake distance, a model which adds predictive power. To do so, let's model price as a linear function, with both a slope and an intercept. The intercept is the price when the lake distance is 0.

$$E(\text{price}|\text{lake.distance}) = \alpha + \beta \text{lake.distance}$$

```
Lake.distance <- c(229, 519, 1037, 1636, 667, 564, 1863, 663, 113, 118)
```

```
plot(Lake.distance,prices)
abline(lm(prices~Lake.distance), lty=2)
```



A regression is a statistical model which estimates the relationship between one variable, called the *dependent variable* and one or more other variables, called the *independent variables*. Regressions require writing down a model, which relates these independent variables to the dependent variable in a formula that describes both the observable and systematic part of the relationship, and the unobservable ideosyncratic component. If we call an observation of the dependent variable y_i , and the observation of the independent variable x_i , we can say they are related in the following linear model.

$$y_i = \alpha + \beta x_i + \epsilon_i$$

Where α and β are called parameters. These are systematic, and apply for all observations. ϵ is ideosyncratic, and is unique to each individual. The purpose of the regression model is to estimate α and β , either to make predictions, or to discover the amount of y that we should expect a change in x to produce:

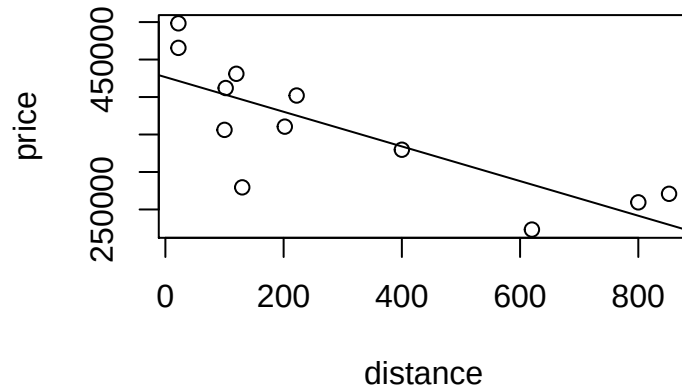
$$\begin{aligned}\frac{dy}{dx} &= \frac{d}{dx}[\alpha + \beta x_i + \epsilon_i] \\ \frac{dy}{dx} &= \frac{d}{dx}\alpha + \beta \frac{d}{dx}x_i + \frac{d}{dx}\epsilon_i\end{aligned}$$

$$\frac{dy}{dx} = \beta$$

Which is consistent with β being the slope of the line produced by the regression. The standard regression practice is to assume a distribution for ϵ , usually $\epsilon \sim N(0, \sigma^2)$. This and the other assumptions of regression will be covered in the following course. For now, there is one that makes the standard linear model inappropriate for our purposes, there is an assumption that the relationship is *linear*.

For example, suppose we get another sample of Condos. We want to predict the value of condo in the South Loop, and we observe the distance to the # 2 bus in feet, and the price of each condo.

```
distance<- c( 22, 120, 400 ,102,22, 620, 100 ,202,222, 130, 800 ,852)
price<-c(465600, 431000, 329700, 412000, 498300, 223200,
        356200, 360500, 402000, 279500, 259500, 270900)
modcondo<-lm(price~distance)
plot(distance, price)
abline(modcondo)
```



<i>Dependent variable:</i>	
	price
distance	-231*** (57)
Constant	426,554*** (23,524)
Observations	12
R ²	0.623
Adjusted R ²	0.585
Residual Std. Error	56,239 (df = 10)
F Statistic	16.517*** (df = 1; 10)
<i>Note:</i> *p<0.1; **p<0.05; ***p<0.01	

This looks like for every foot that the condo is away from the bus stop, the price declines by about \$231. Our best guess for the price of a condo that is 600 feet away from the bus is about $426000 - 231 \times 600$

Suppose, rather than the price, we just had a categorical outcome, like the bigchange variable from above? Lets say rather than the price, we just had an indicator for whether my real estate agent would show it to me, after being told that I need to use the bus. Lets say that we have a 1 if the real estate agent shows it, and a 0 if he does not. Here I show both the linear model, as well as the prediction from that linear model if the line falls above .5 or below .5.

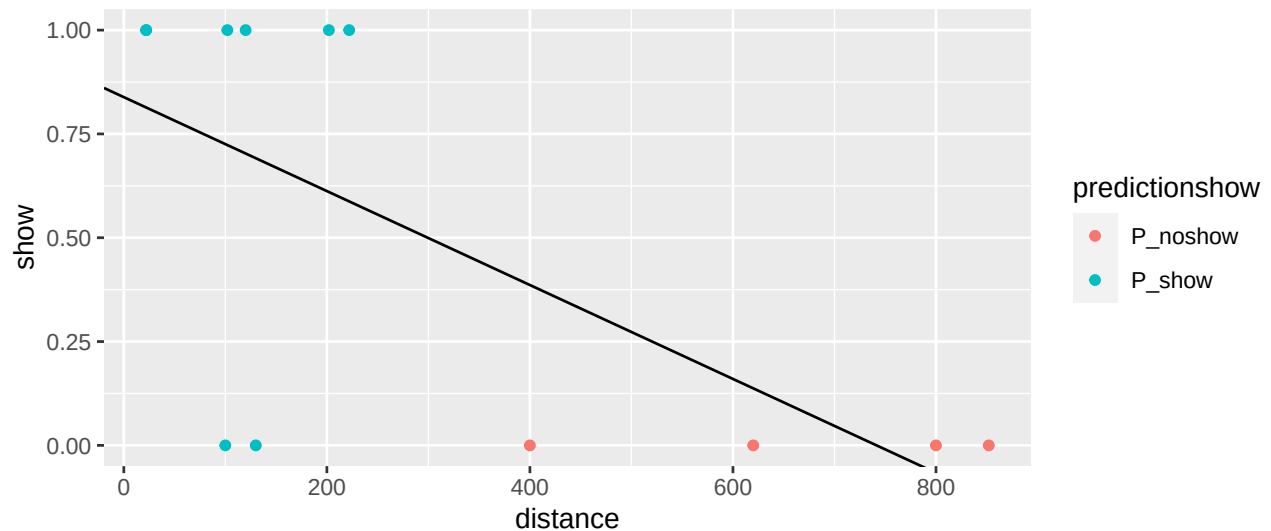
```
distance<- c( 22, 120, 400 ,102,22, 620, 100 ,202,222, 130, 800 ,852)

RealEstateAgent <- c("show", "show", "don't show", "show", "show",
                    "don't show", "don't show", "show", "show",
                    "don't show", "don't show", "don't show")

show <- ifelse(RealEstateAgent=="show", 1 , 0)

modshow <- lm(show~distance)
modpredictions <- data.frame(distance, show,
                             predictionshow= ifelse(predict(modshow)>.5, "P_show","P_noshow"))

coefmodshow <- coef(modshow)
ggplot(modpredictions, aes(distance, show, color=predictionshow) )+
  geom_point()+
  geom_abline(intercept=coefmodshow[1], slope=coefmodshow[2])
```



This model works just fine, but the misses above and below the line are quite uneven, and the line never really makes a sensible prediction for any distance.

The solution is to not fit a straight line. Rather, we will use a transformation of the linear model. Below, we introduce a broad outline of the use and some of the properties of such a model.

```
distance<- c( 22, 120, 400 ,102,22, 620, 100 ,202,222, 130, 800 ,852)

RealEstateAgent <- c("show", "show", "don't show", "show", "show",
                    "don't show", "don't show", "show", "show",
                    "don't show", "don't show", "don't show")

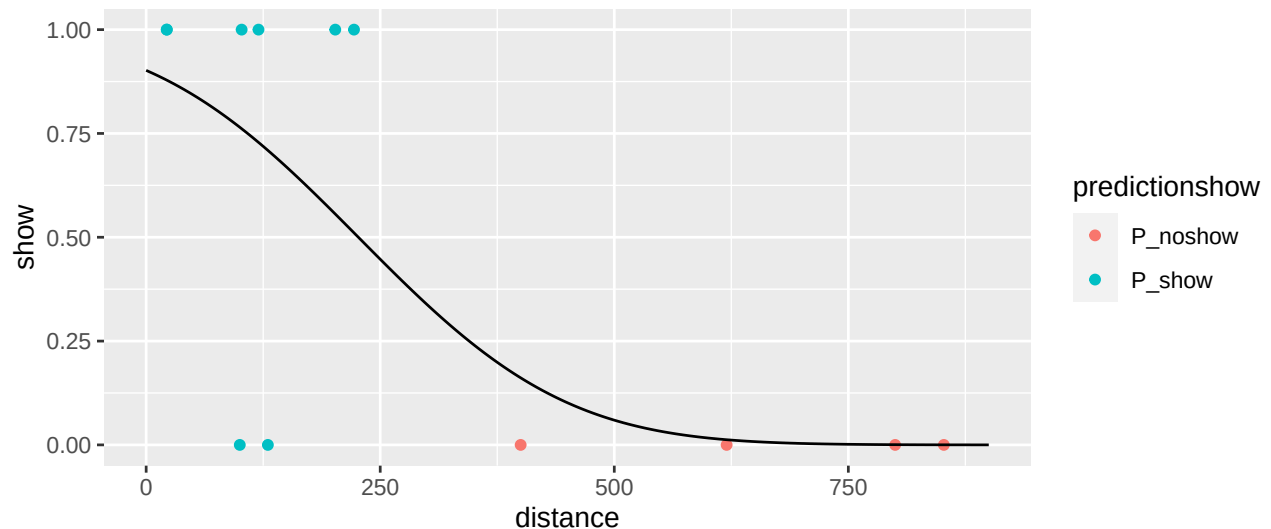
show <- ifelse(RealEstateAgent=="show", 1 , 0)

modshowprobit <- glm(show~distance, family=binomial(link="probit"))

modpredictions <- data.frame(distance, show,
                             predictionshow = ifelse(predict(modshowprobit, type="response") >.5,
                                                         "P_show", "P_noshow"))

simulatelatent <- data.frame(distance=0:900,
                             predictmod = predict(modshowprobit,
                                                    newdata=data.frame(distance=0:900),
                                                    type="response"))

ggplot(modpredictions, aes(distance, show) )+
  geom_point(aes( color=predictionshow))+
  geom_line(data=simulatelatent,
            aes(distance, predictmod))
```



##Probit for categorical dependent variables

In the case of the reliability of the religion measurements, the dependent variable is binary, the independent variable is a 3 category vector, which we can indicate as follows:

```
probitmod <-  
  glm(bigchange ~ reliab,  
      data = religions,  
      family = binomial(link = "probit"))
```

Implicitly the computer takes `reliab`, and splits it into two category indicators. The first is the omitted reference category, the other two are entered linearly. You could force the model to do the same thing explicitly as well, using the `I(x == "category")`, which takes on 1 if x takes on that category and 0 otherwise:

```
probitmod <-  
  glm(  
    bigchange ~ I(reliab == 3) + I(reliab == 2),  
    data = religions,  
    family = binomial(link = "probit")  
  )
```

The formula can be described notationally as follows:

$$Pr(y_i = 1|x_i) = \Phi(\alpha + \beta_{low}I(x_i = 'Low') + \beta_{medium}I(x_i = 'Medium'))$$

You read this as the probability that y is equal to 1 given x is equal to the standard normal CDF evaluated at α plus β_{low} if the observation is low reliability, α plus β_{medium} if the observation is medium reliability, and just α if the observation is high reliability. We will learn more about the normal CDF later in this course, but suffice it to say that it is an S or backward S shaped curve. You can explore it in R using the `pnorm()` command.

Just as with a linear model, the coefficients can be returned either directly or in a summary command.

```
coef(probitmod)
```

```
##          (Intercept) I(reliab == 3)TRUE I(reliab == 2)TRUE
##          -1.61687015      -0.36723522      -0.09783118
```

```
summary(probitmod)
```

```
##
## Call:
## glm(formula = bigchange ~ I(reliab == 3) + I(reliab == 2), family = binomial(link = "probit"),
##      data = religions)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -0.3299  -0.2972  -0.2972  -0.2972   2.7370
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -1.61687    0.09362  -17.271  <2e-16 ***
## I(reliab == 3)TRUE -0.36724    0.19494   -1.884   0.0596 .
## I(reliab == 2)TRUE -0.09783    0.11267   -0.868   0.3852
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 709.02  on 1994  degrees of freedom
## Residual deviance: 705.16  on 1992  degrees of freedom
## AIC: 711.16
##
## Number of Fisher Scoring iterations: 6
```

The difference is that the coefficients are on the latent, non-linear cumulative normal distribution. To interpret them, we can pass it through R's `cdf` function for the normal.

Suppose we want the probability of a big change when the reliability of the data is high. This is just the intercept, α , as the other two coefficients are set to 0. Reading the regression table, we see that $\alpha = -1.61687$. We pass the value into the `pnorm` function, which is R's calculator for the normal CDF.

```
pnorm(coef(probitmod)[1])
```

```
## (Intercept)
## 0.05295316
```

If the the reliability is medium, then we add the intercept to the coefficient on medium, which is the third element in the `coef` command, and then pass the result to the `pnorm` command:

```
# pnorm(coef(probitmod)[1] + coef(probitmod)[3])
pnorm( - 1.61687 -0.09783)
```

```
## [1] 0.04320012
```

If we compare these to the table of proportions, we see the similarity. Here I round to the nearest fourth digit.

```
round(prop.table(table(religions$reliab, religions$bigchange), margin=1),4)
```

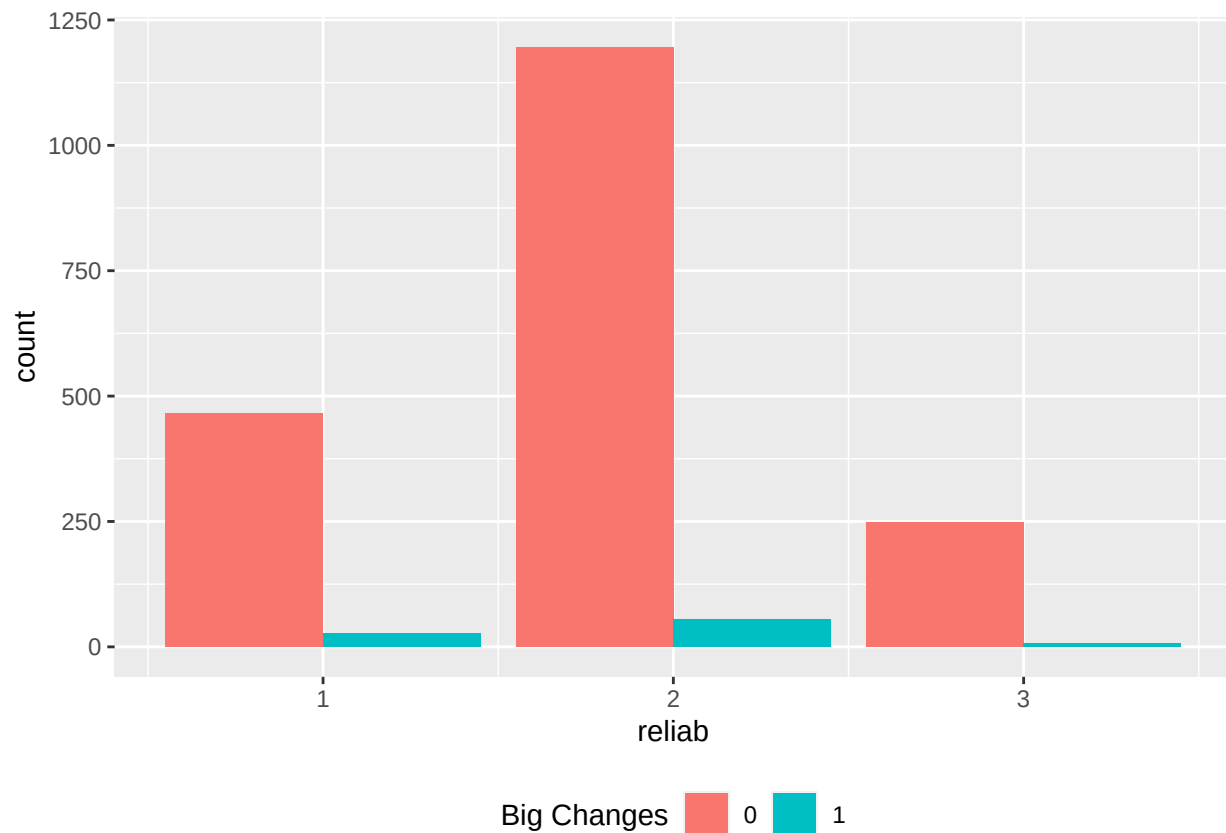
```
##
##      0      1
## 1 0.9470 0.0530
## 2 0.9568 0.0432
## 3 0.9764 0.0236
```

The second row, second column indicates that among medium reliable data, 4.32 percent experience a big change. This is exactly the interpretation we can give to the tranformed coefficient from the probit model. You can read more about these and other related models in J. Scott Long's **Regression Models for Categorical and Limited Dependent Variables** or in a course on Maximum Likelihood Estimation (MLE).

Other than tables, a useful way to present discrete data is with bar charts. Try each of the following commands

```
# ggplot(religions, aes(reliab))+  
# geom_bar()  
  
# ggplot(religions, aes(reliab, group=as.factor(bigchange), fill=as.factor(bigchange)))+  
# geom_bar()  
  
# ggplot(religions, aes(reliab, group=as.factor(bigchange), fill=as.factor(bigchange)))+  
# geom_bar(position="dodge")  
  
ggplot(religions, aes( reliab,  
  group = as.factor(bigchange),  
  fill = as.factor(bigchange))) +  
geom_bar(position = "dodge") +  
guides(fill = guide_legend(title = "Big Changes")) +  
theme(legend.position = "bottom")
```

Don't know how to automatically pick scale for object of type haven_labelled/vctrs_vctr/double. Defaulting to numeric scale.



Problems

- 1) Using the world religion dataset, what country experience the largest percentage increase in Islam over the dataset?
- 2) Go to ICPSR, sign up, download the China Multi-Generational Panel Dataset, Shuangcheng (ID 35292). What is the number of sampled individuals in 1867 that Absconded (escaped from indentured servitude)? What was their average age?
- 3) Suppose we want to study the association between religiosity and civil war. Be sure to specify both the independent and dependent variables. Write and present a table that combines information from the religiosity dataset and the following dataset on civil war. Be creative. There are two ways to study civil war, the first is whether or not that country is experiencing a civil war (war), and the other is the onset of such a war (onset). The main predictor of being in a civil war this year is having been in one last year, so some people restrict their study to onset.

```
fldta <- read_dta("FearonLaitin.dta")
```

```
fldta <- fldta %>%  
select(  
  cowcode = ccode,  
  year = year,  
  cname = cname,  
  onset = onset,  
  war = war,  
  polity2 = polity2,  
  Oil = Oil,  
  GDPpercap = gdpen  
)  
# str(fldta)
```